CACHE ARCHITECTURE IN MODERN PROCESSORS

Why we need caches

DRAM latency is too high relative to CPU speed

Component	Latency
Registers	~ 1 cycle
L1 cache	~ 4 cycle
L2 cahce	~ 10 cycle
L3 cache	~ 30-50 cycle
DRAM	~ 100-200 cycle

Without caches CPU would stall most of the time

Caches act as small fast memories storing recently used data

Caching

Modern CPUs have separate caches for data and for instructions.

In a pipelined datapath it is possible in the same clock cycle:

- fetch an instruction from the Instruction cache (I-cache)
- load/store data (for a different instruction) from/to the data cache (D-cache)

I-cache and D-cache are together the first-level cache (L1)

Most processors have a second-level cache (L2) and there are processors with a third-level cache (L3)

The Cache Block Concept

Caches do not store single bytes or words, they store block (cache lines)

Typical block size:

- 32 B
- 64 B
- 128 B

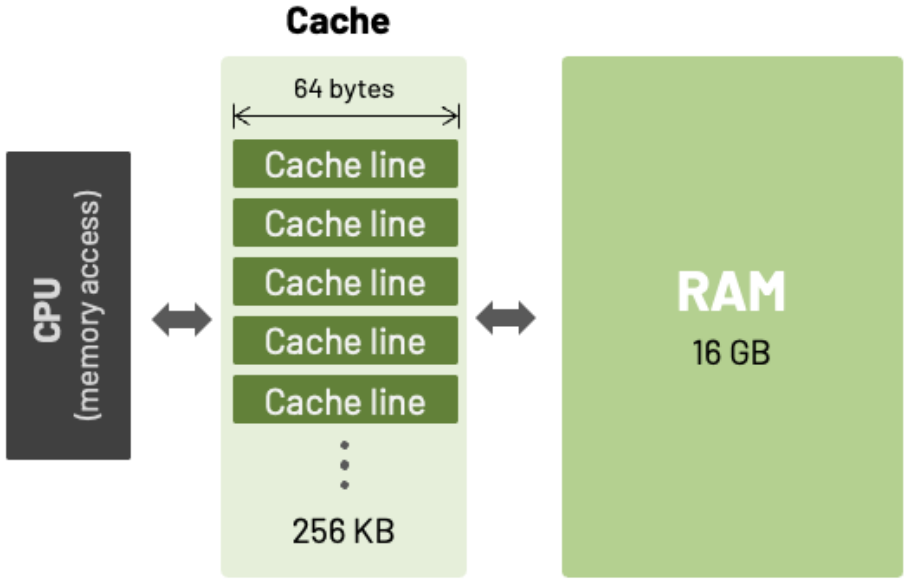
Why ?

To exploit spatial locality

Basic Cache Operation

When CPU requests and address

1. Cache checks if block is present
2. If present -> cache hit
3. If absent -> cache miss
4. Block fetched from next memory level



Miss penalty = time to fetch a block from lower level

Cache Organization Problem

Where should a memory block be placed in cache?

Three main design exists:

1. Direct-mapped
2. Set-associative
3. Fully associative

Each represents a different trade-off

Direct-Mapped Cache

The most simply caches.

A direct-mapped cache consists of 2^n entries (sets)

Each entry stores a block -> 2^m consecutives bytes (typically 32 or 64) of RAM

Each entry has:

1. A validity bit for the entry
2. A tag that uniquely identifies the block stored in the cache with respect to the RAM

Direct-Mapped Cache

Each memory block maps to exactly one location.

To find the cache location containing the RAM block holding the addressed data or instruction, the computation is

$$Cache\ index = (Block\ address) modulo (Number\ of\ sets)$$

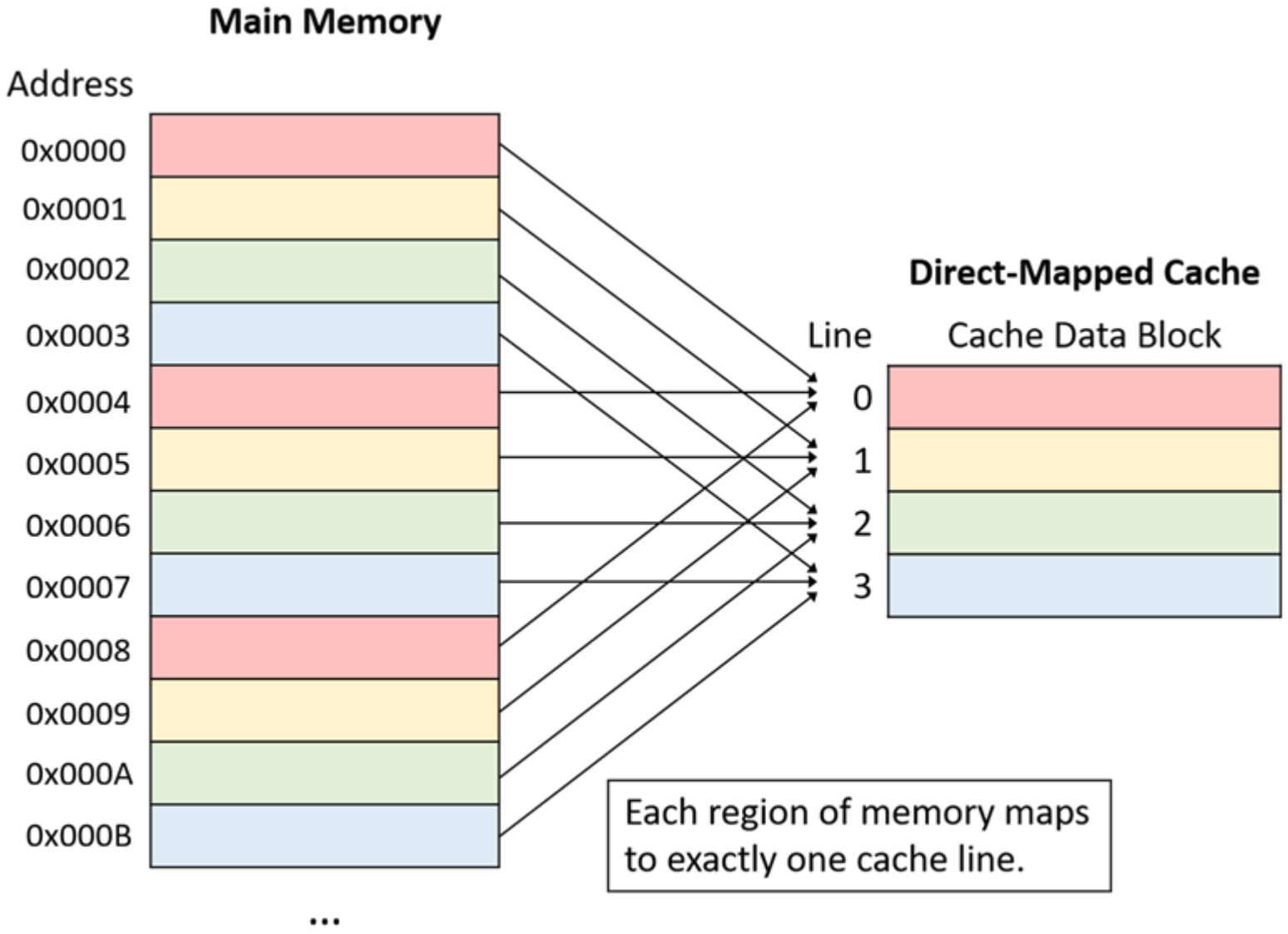
Advantages:

- Very simple
- Very fast
- Low energy

Disadvantages:

- High conflict misses

Direct-Mapped Cache



Direct-Mapped Cache

To locate the data in a cache, the address must be divided in:

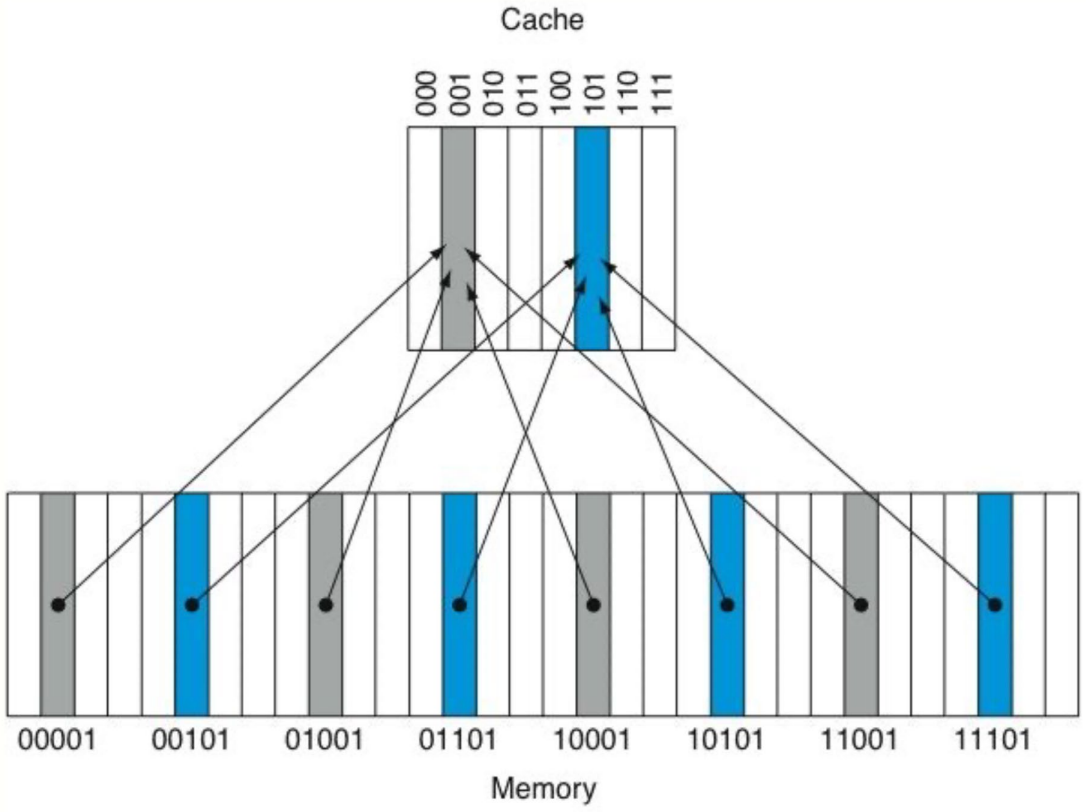
- Tag -> verifies correct block
- Index -> selects cache set
- Offset -> selects byte inside block

Example: 32 bits address, cache size of 64 KB and block size of 64 B

Block size = 64 B -> offset 6 bits
Number of blocks -> 64 KB / 64 = 1024
Index bits -> $\log_2(1024) = 10$
Tag bits -> 32 – 10 – 6 = 16

Direct-Mapped Cache

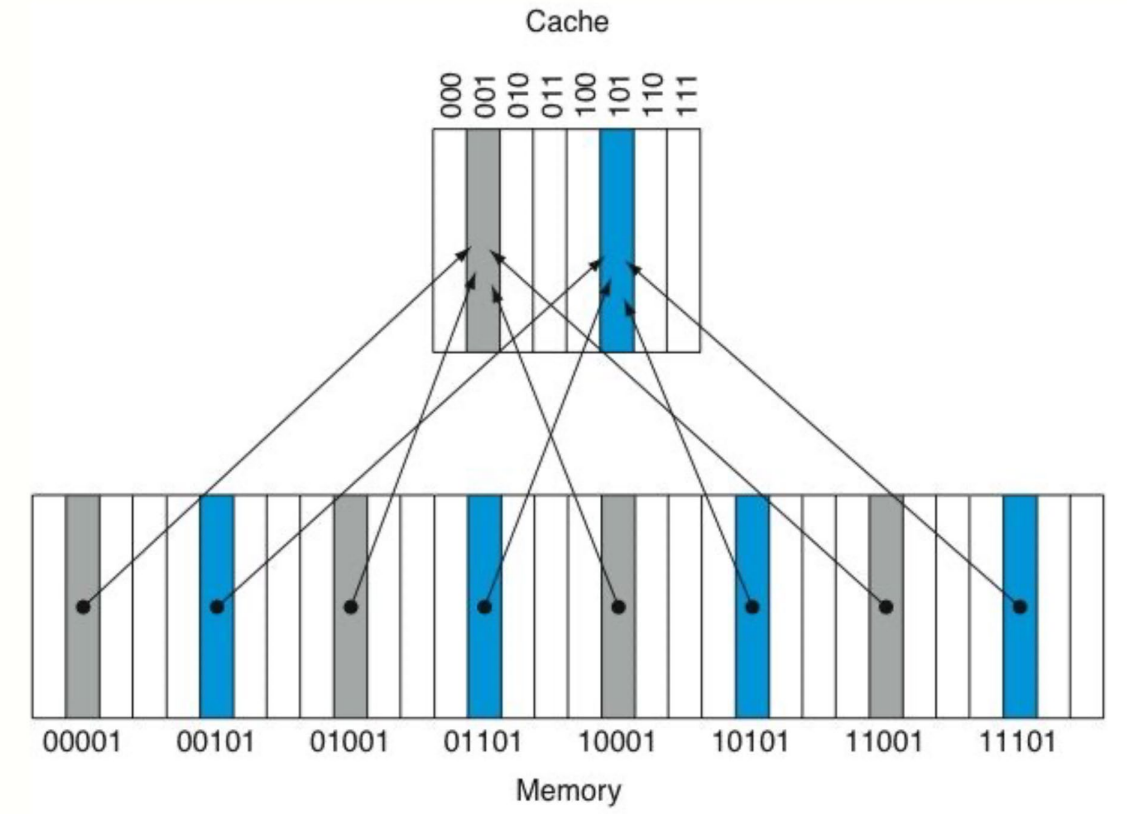
To better understand the tag meaning, consider a RAM split into 32 blocks and an 8-entry cache



The address 00001, 01001, 10001 and 11001 will all match the cache entry 001!

The blocks stored in each entry are discriminated by the tag associated to each cache line.

Direct-Mapped Cache



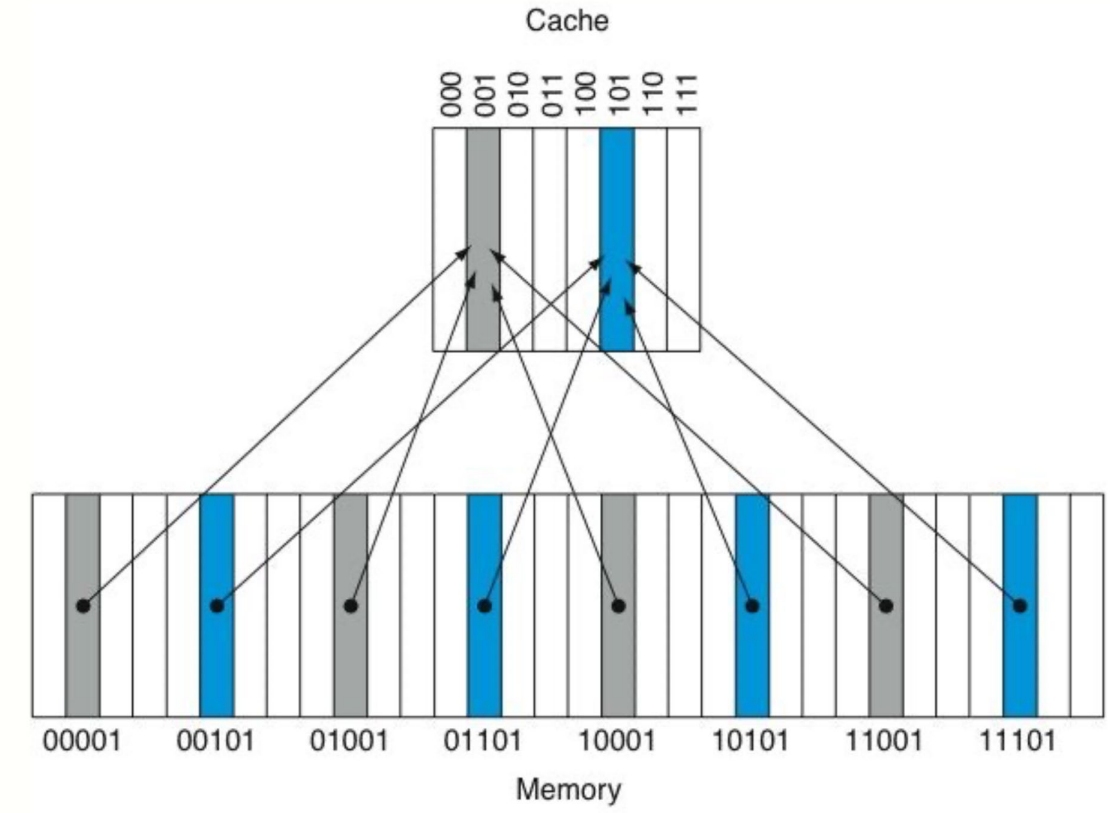
5 bits address

8 cache entries -> 3 index bits

No offset since we have that each block stores a single byte

Tag bits -> 5-3 = 2

Direct-Mapped Cache

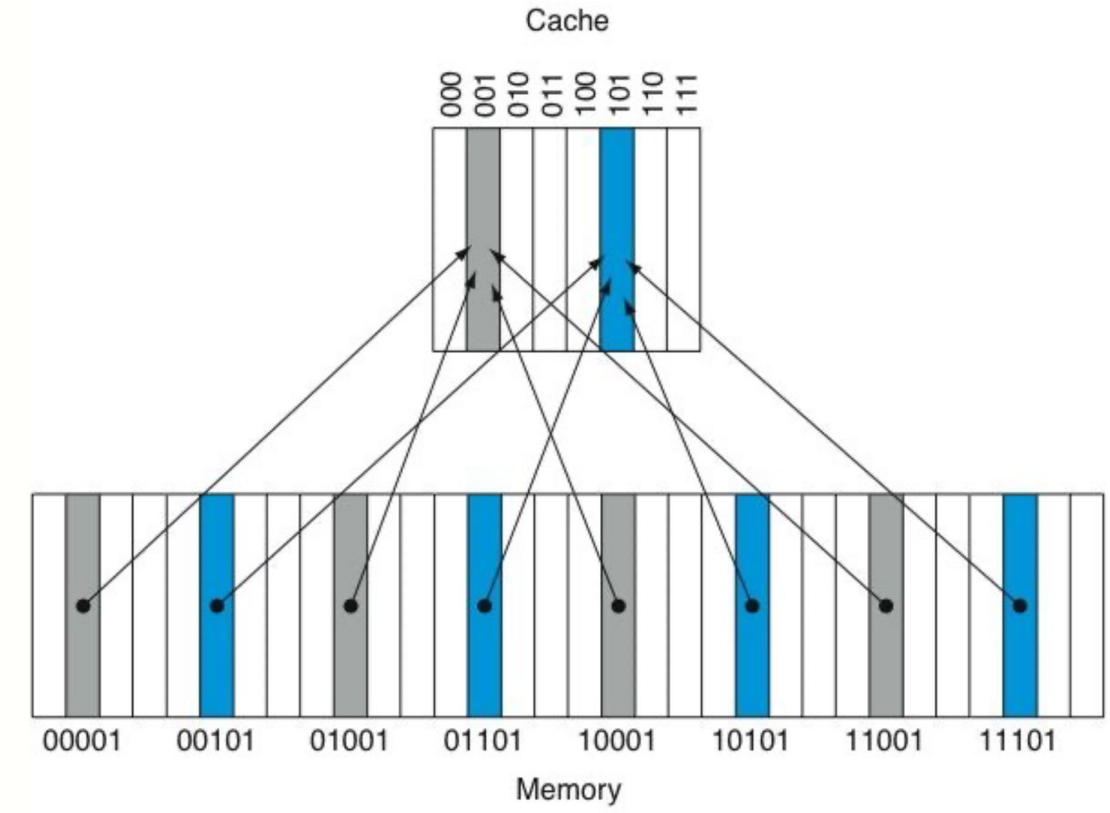


The CPU requests block 01001. It checks if cache entry 001 has it validity bit set

If the bit is set, entry 001 tag is compared with the two most significant bit of the address

If there is a match, cache entry 001 holds the requested line, otherwise there is a cache miss

Direct-Mapped Cache



If the entry is not valid or the tags do not match, there is a cache miss

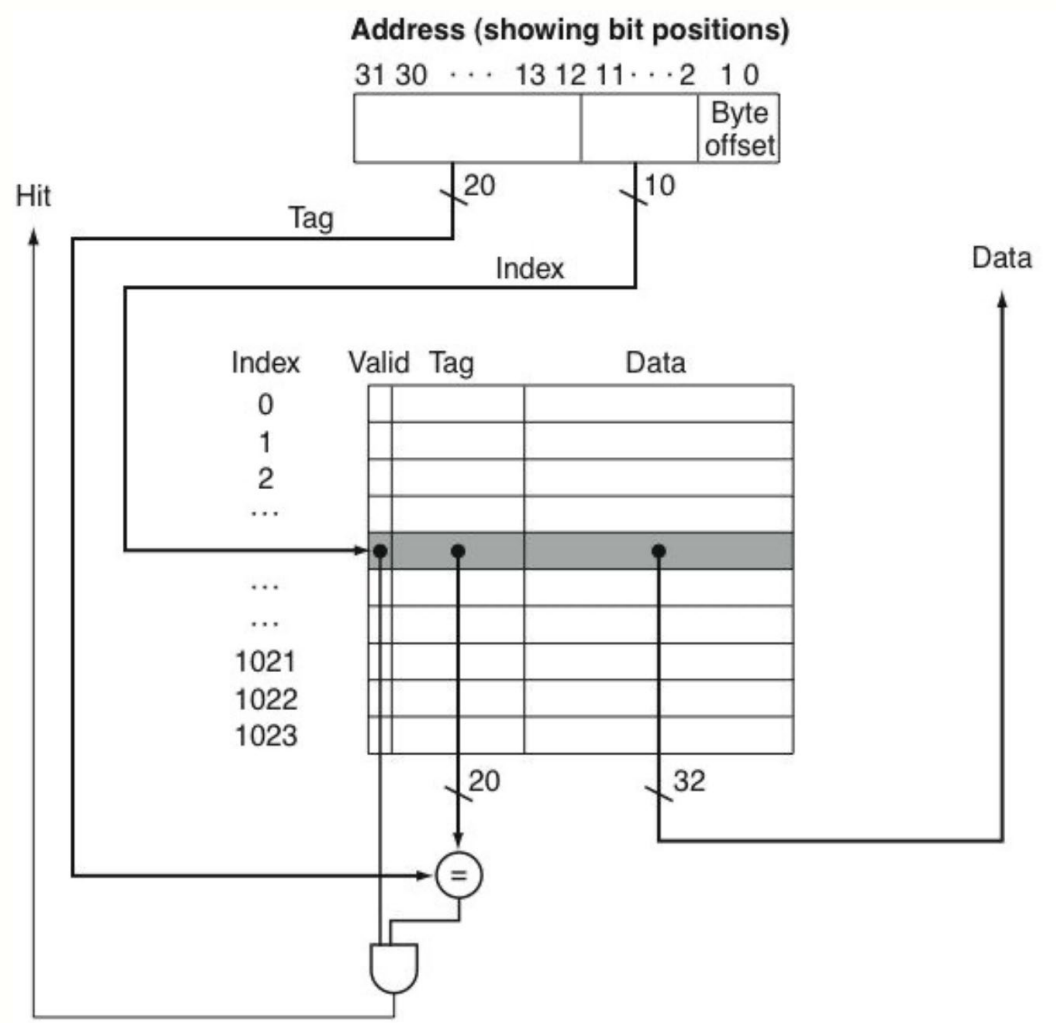
The missing block is fetched from RAM and it replaces the one present

Direct-Mapped Cache

Consider a direct-mapped cache with 1024 4 B entries.

The addresses are 32 bits.

Block size = 4 B -> offset 2 bits
Index bits -> $\log_2(1024) = 10$
Tag bits -> 32 – 10 – 2 = 20



Direct-Mapped Cache

The effective size of a cache is larger than the nominal one since it must store all the information to correctly identify the RAM block stored

In the previous case, each cache line stores:

1 validity bit
20 bits for the tag
32 bits for the data

for a total of 53 bits.

Nominal size -> 1024 lines × 4 B = 4096 B = 4 KB

Effective size -> 1024 lines × 53 bits = 54272 bits ≅ 6.6 KB

Set-Associative Cache

Born as a solution to conflict misses

Allow multiple blocks per index

Each index corresponds to a set

Block can occupy any way inside the set.

Example:

- 2-way
- 4-way
- 8-way

Set-Associative Cache

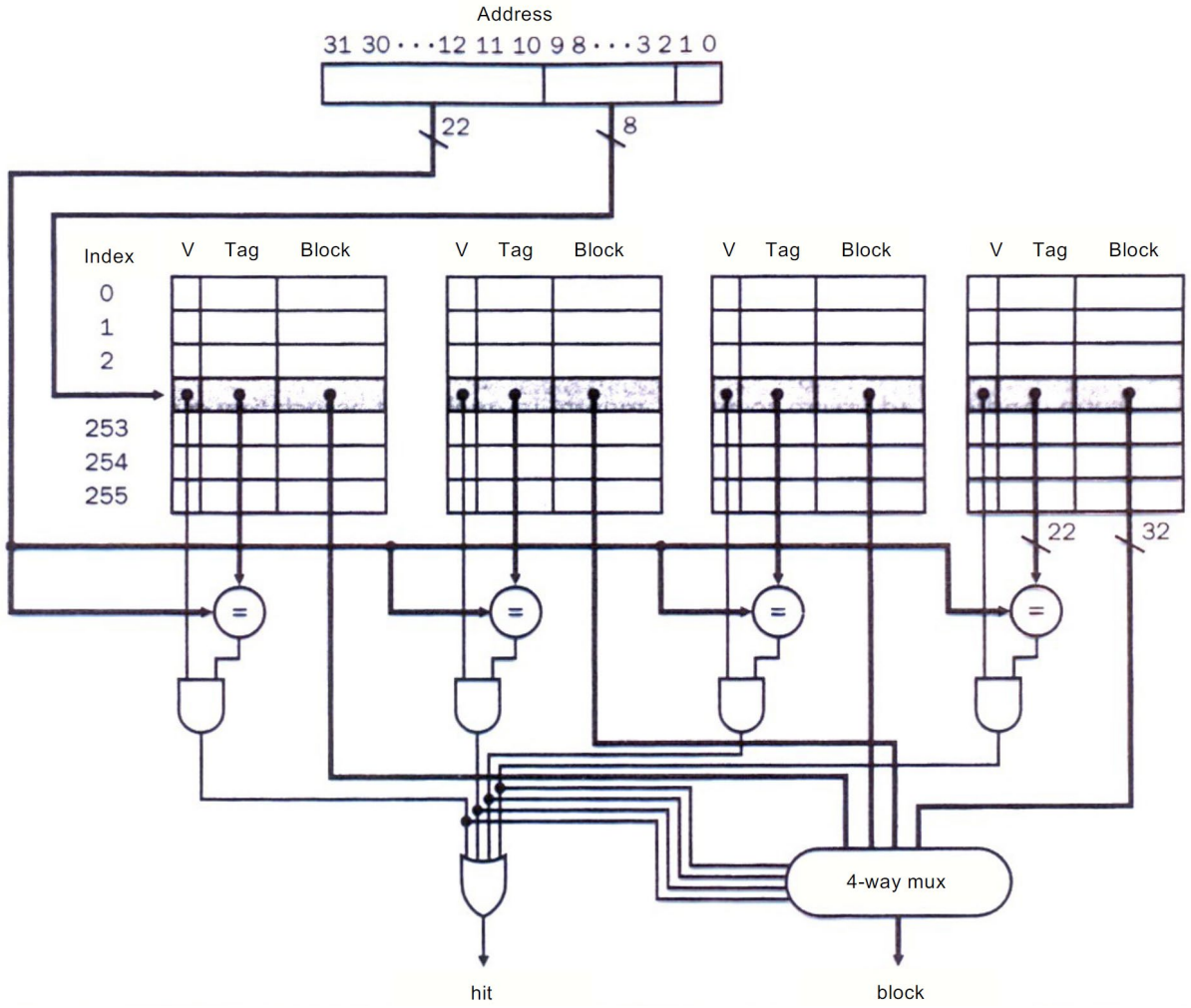
Operations:

1. Index selects a set
2. All tags in set compared in parallel
3. Matching tag indicate hits

Trade-off is more hardware complexity than directly-mapped cache

Set-Associative Cache

Structure of a 4-way set-associative cache with 256 sets (8 bit index)



Fully Associative Cache

Most flexible design

Block can be placed anywhere in cache

No index

All tags are compared simultaneously

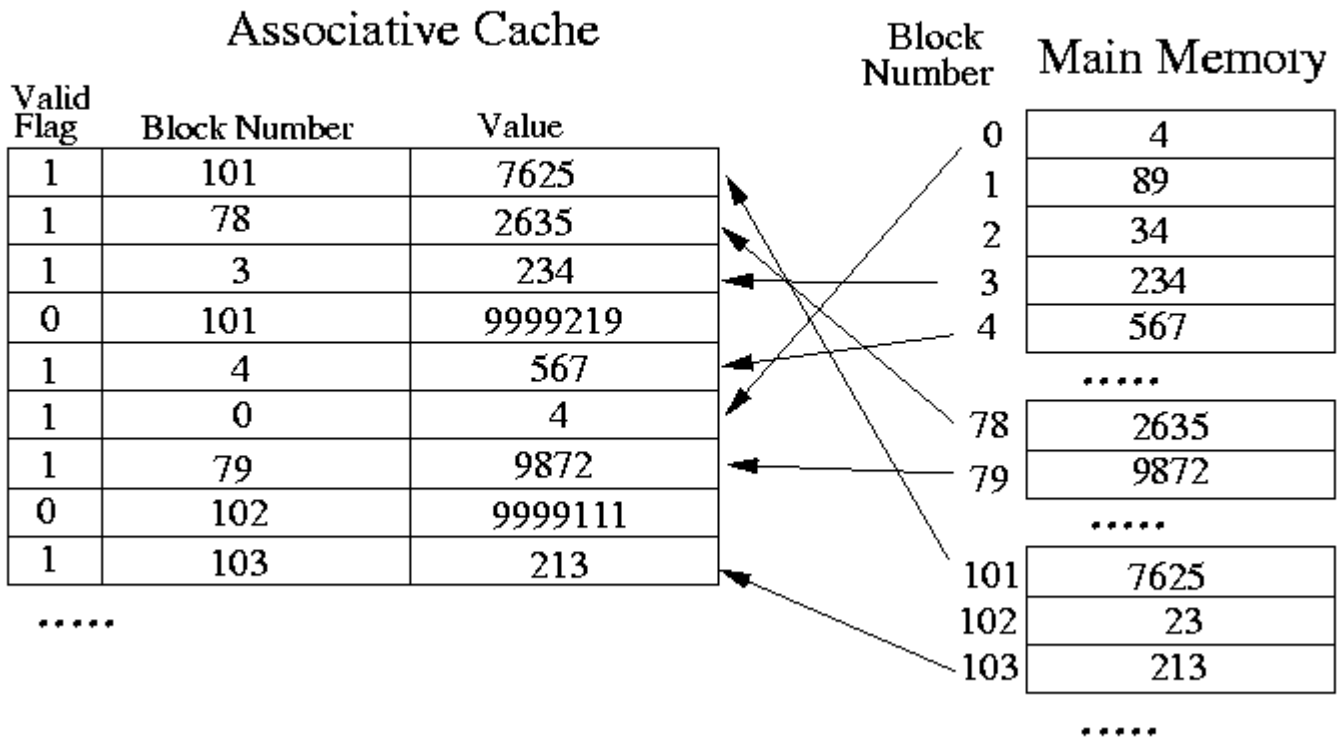
Advantages:

- Eliminates conflict misses

Disadvantages:

- Expensive
- Slow for large caches

Fully Associative Cache



Associativity Trade-Off

Advantages of increasing associativity

- Fewer conflict misses
- Better cache utilization

Disadvantages of increasing associativity:

- Longer hit time
- Higher energy
- More complex hardware

Associativity Trade-Off

Engineering reality

Level	Associativity
L1	4-8 ways
L2	8 ways
L3	12-16 ways

Block Size Trade-Off

Block size (typical 64 B in modern CPUs) influences:

- Spatial locality
- Miss penalty
- Bandwidth usage

Small blocks:

- Lower miss penalty
- Less bandwidth waste

Large blocks:

- Better spatial locality
- Higher miss penalty

Interaction with DRAM

Block size must match DRAM behavior

DRAM accesses entire rows efficiently

Fetching blocks aligns with

- Memory burst transfers
- Bus width

Cache block size is also a memory system design choice

Real Processor Example

Intel Core architecture typical hierarchy:

- L1: 32 KB, 8 way
- L2: 256-512 KB, 8 way
- L3: several MB, 12-16 way

Design goals:

L1 -> minimal latency
L3 -> maximize capacity

Different levels optimize different metrics

Key Engineering Trade-Offs

Cache design balances:

- Hit time
- Miss rate
- Power
- Area
- Scalability

These trade-offs drive real processor design

Closing

Summary of key concepts:

- Cache block
- Address decomposition
- Cache mapping strategies
- Associative trade-offs
- Block size design

Next episode: cache performance optimization (miss types, replacement policies, write policies, non-blocking caches)